Challenge 2: Reconstruction and Optimization of a 256-bit Permutation

Crypto Contest Submission

23 July 2026

1 Summary

The recovered one-round permutation is

$$\text{rotate} \rightarrow \text{XOR} \rightarrow \text{reverse all 32 bytes} \rightarrow \text{add},$$

with word rotations

$$\texttt{rot = \{43, 7, 29, 14\}}.$$

The final implementation passes all 1,000 supplied one-round vectors, the supplied 20-round vector, and 100,000 deterministic randomized differential tests. The first optimization replaces the generic byte map with four 64-bit byte swaps. The final optimization composes rounds in pairs: the word reversal cancels after two rounds, exposing four independent scalar dependency chains. Ten pairs are fully unrolled, and a local BMI2 target lets GCC use non-destructive constant rotates under the supplied compile command. The same source has an adaptive score build: global BMI2 changes the core to `always_inline`, and a larger inline threshold integrates the public wrapper into the timing loop. Two repeated campaigns measured paired speedups of 1.116x and 1.118x over the default build. A later lane-wise AVX2 candidate reduces the exact score loop from 322 scalar instructions to 122 vector instructions, but AMD affinities disagree on whether it beats scalar. It is therefore a high-priority 255H experiment while the scalar source remains the submission incumbent. Follow-up split-width, 113-build code-generation, and page-aligned three-timer experiments found no new winner and localized the AMD sign reversal to unstable scalar throughput on the shared host rather than a different binary or timer. An eighth pass reduced the AVX2 loop from 579 to 569 bytes without adding instructions or memory, but two 32-sample AMD campaigns measured a statistical tie. A ninth pass used commutative VEX source placement to reach 122 instructions, 549 bytes, and no hot memory, and retained a 136-byte compact block alternative. Schema 5 also verifies the exact final state after every timed process; it rejects a deliberately halved loop that the older public-function and assembly gates misreported as 2.253x faster. Fresh 32-sample AMD campaigns put every new candidate's interval across 1. These smaller streams are therefore target-only 255H candidates, not new submission incumbents. A tenth pass replaced the intrinsic compact blocks with exact counted-assembly front ends: block2, block3+tail1, and block5 now occupy 122/238/292 bytes and execute 133/129/127 modeled non-padding instructions. Register placement, boundary placement, scalar destructive-rotate, and SHLD screens produced no static target winner. The timing schema now also binds the reported average to total elapsed time, checks timed-work child-CPU coverage, and challenges each source at a nonce-derived alternate iteration count. An eleventh pass adds a raw-sample stationarity promotion gate: CPU 1's AVX2 pairs remain eligible but tied, its scalar comparison is diagnostic-only, and the entire CPU 3 campaign is diagnostic-only. A grammar-scoped three-instruction AVX2 synthesis search and exact split-shuffle controls found no smaller transform, while LLVM 19's tested target labels all selected the same scheduling metrics and do not supply Core Ultra 7 255H evidence.

2 Recovering the structure

Every component operation is invertible. Addition is inverted by subtraction modulo 2^{64}, XOR is self-inverse, and the map `shuffle_map[i] = 31-i` is also self-inverse. For every supplied input/output pair, I computed

```
t0 = output - constants1 // modulo 2^64 per word
t1 = reverse_all_32_bytes(t0)
t2 = t1 XOR constants2
```

and tested each permitted $r \in \{1, \ldots, 63\}$ for

```
ROTL64(input[i], r) == t2[i].
```

The recovery script intersects the candidate sets from all 1,000 vectors rather than trusting one possibly degenerate word. Each word has one candidate: 43, 7, 29, and 14. Forward evaluation confirms the stated operation order on all supplied vectors.

3 Reference and first specialization

The direct reference calls the supplied operations in the recovered order:

```
rotate_words_left_64wise(state, rot);
xor_constants_256wise(state, constants2);
shuffle_bytes_256(state, shuffle_map);
add_constants_64wise(state, constants1);
```

The fixed map is a complete byte reversal, not an arbitrary permutation:

```
reverse32(x0,x1,x2,x3) =
    (BSWAP64(x3), BSWAP64(x2), BSWAP64(x1), BSWAP64(x0)).
```

Thus 32 indexed byte copies and two temporary arrays become four `__builtin_bswap64` operations. One specialized round is

```
y0 = BSWAP64(ROTL64(x3,14) ^ C2[3]) + C1[0]
y1 = BSWAP64(ROTL64(x2,29) ^ C2[2]) + C1[1]
y2 = BSWAP64(ROTL64(x1, 7) ^ C2[1]) + C1[2]
y3 = BSWAP64(ROTL64(x0,43) ^ C2[0]) + C1[3]
```

The constant shift counts permit immediate-count rotate instructions. The pre-optimization submission called a one-round, state-writing helper from the provided loop twenty times. A later register-resident twenty-round loop served as an experimental baseline.

4 Two-round composition

A round reverses the four word positions. Two reversals restore the original positions, so each two-round output depends only on the word at the same index. Let T_j denote the rotate, XOR, byte-swap, and destination addition associated with source word j. Two rounds are

```
x0 <- T3(T0(x0)) x1 <- T2(T1(x1))
x2 <- T1(T2(x2)) x3 <- T0(T3(x3)).
```

This form exposes four independent chains without round-by-round word moves. The helper expands this block ten times explicitly. GCC's local `target("bmi2")` attribute permits `RORX`, a non-destructive rotate that does not update condition flags, while the rest of the contest file still compiles with the supplied default command.

The helper attribute is conditional on `__BMI2__`. With the plain command it remains the aligned local-target noinline helper. With `-mbmi2 -finline-limit=2000`, caller and callee target

options match, the core is forced into the public function, and GCC also inlines that function into `main`. This preserves a no-extra-flags compatibility path while enabling the faster path permitted by the statement's optimization-flag rule.

The statement permits external helper functions and restricts edits inside the provided 20-round loop. On its first iteration, that permitted line calls the helper for all twenty rounds and sets the supplied loop counter to 19. The helper applies the same permutation to any input state; it contains no benchmark-input or output hard-coding.

5 Candidate experiments

The experimental harness builds each candidate into a separate binary so that large unrolled functions do not alter one another's instruction-cache layout. Every candidate first passes the supplied vectors and 100,000 randomized differential cases. On an AMD EPYC 7B12 VM with GCC 12.2.0, `-O3 -march=native`, CPU 2, and 15 repeated samples, the screening result was:

Candidate	Median ns	MAD ns	vs. pre-opt.
Preserved pre-optimization form	41.894	2.834	1.000x
Register loop	40.655	3.083	1.031x
Paired AVX2 loop	39.718	0.131	1.055x
Paired scalar loop	37.751	2.649	1.110x
Paired AVX2 full unroll	39.417	0.086	1.063x
Paired scalar full unroll	36.824	2.617	1.138x
Paired scalar unroll + BMI2	36.249	3.195	1.156x
One-state AVX2	73.630	0.266	0.569x

The one-state AVX2 version is slower because each dependent round requires variable left and right shifts, OR, XOR, byte shuffle, half permutation, and addition. Four scalar words instead provide four shorter independent chains. A transposed four-state AVX2 version improves throughput, but the official API updates one state serially and cannot use that batching. The paired AVX2 loop was stable but slower than scalar BMI2 on this host. A non-BMI2 inline unroll was nearly tied with BMI2 in the contest-shaped test, so both remain useful target-machine controls.

5.1 Deep frontend and cache review

An additional 18-candidate sweep varied unroll factor, 16/32/64/128-byte alignment, literal constants, inlining, wrapper layout, SIMD, and chain scheduling. The submitted helper is 1,267 bytes, has no hot-body spills, and keeps four state words and eight constants in registers. Its four independent `RORX/XOR/BSWAP/ADD` chains run close to the simple 80-cycle dependency depth, so the remaining difference is mainly frontend and code layout.

A 715-byte half-unrolled loop measured 1.0867x in one paired session, but only 0.9712x in an immediate repeat and 0.9214x with `-march=native`. Alignment changes, embedded constants, and forced inline were neutral or slower; serialized chains reached only 0.7305x. Under the later cross-call inline flags, full/pair-loop/half-unroll medians were 37.279/38.618/38.727 ns. The full unroll therefore remains the arithmetic core.

5.2 Cross-call inlining

With only `-mbmi2`, GCC merges the private core into the public `permute_20rounds`, but `main`'s timing loop still calls that function. Adding `-finline-limit=2000` removes this second boundary. Assembly inspection found no core-external call, push/pop, or state/constant memory operand in

the timed loop; all twelve values remain in registers across outer iterations. A default/`-mbmi2`/full-inline ablation measured paired 1.000x/0.999x/1.124x, identifying the public-to-main boundary as the gain. GCC 12 generated byte-identical binaries with inline limits 700 and 2000. A later digest-pinned official GCC 13.3.0 build confirmed that the two complete binaries are also byte-identical in the target compiler. Explicit 32/64/128-byte alignment moved the inlined loop entry but measured paired 1.006x/0.991x/0.990x; all bootstrap intervals touched or crossed 1, so the default alignment remains. Earlier generic IRA/native controls also failed their confidence gates. Thus the recommended score command keeps neither extra flag without a real-255H confirmation.

The repository assembly audit locates the timed backedge between the final two `clock()` calls. Both inline limits produce the same normalized 1,216-byte, 322-instruction loop: 80 each of RORX, XOR, BSWAP, and ADD/LEA, with no call, push/pop, or state/constant memory operand. A portable forced-inline control reduced total text by 292 bytes and the loop to 1,060 bytes, but destructive ROL/ROR added a move and measured only 0.985x (95% CI 0.958–1.002x). Smaller code did not beat BMI2.

5.3 Second-wave alternatives

Byte swap commutes with XOR, so $B(u \oplus K) = B(u) \oplus B(K)$; moreover, XOR with 2^{63} equals addition of 2^{63} modulo 2^{64}. This allowed a pre-swapped constant and one-bit absorption, but did not remove an instruction. An apparent 1.095x–1.116x separate-process gain disappeared under same-process AB/BA timing and reversed function layouts (1.0146x and 0.9943x). Handwritten inline assembly measured 0.9975x.

Conjugating the opposite-phase transforms allowed two 2-lane SIMD chains and reduced the generated core from roughly 320 to 240 instructions. It still measured only 0.845x–0.848x because four independent scalar chains became two longer vector chains and each rotate required shift/shift/OR.

An exact 64-bit unary-function table needs 128 EiB per function, or 256 EiB for the two conjugacy classes. Empirical influence and mixed-derivative checks also rejected byte-independent XOR/add table decomposition: all 64 input/output-byte edges appeared, all 1,000 mixed XOR derivatives were nonzero, and a restricted 16-dimensional ANF reached degree 16 in 29 of 64 output bits. These observations do not prove a general circuit lower bound, but directly falsify the proposed small-table forms. Compiler scheduling, target clones, PGO/LTO, manual BMI2 scheduling, and cross-call batching likewise produced no portable valid winner; batching is incompatible with the serial state recurrence.

For the actual constants, exhaustive low-bit projections also reject moving XOR through ADD: the equation $(x \oplus C) + A = (x + B) \oplus E$ has no solution already at widths 5, 3, 10, and 4 for the four transforms. Folding bit 63 into ADD works for two constants but leaves a nonzero XOR mask. The four two-round linear bit permutations match neither one rotate nor one rotate/byte-swap composition. Exact alternatives measured 0.980x for post-BSWAP XOR, 0.983x for top-bit folding, 0.620x for arithmetic XOR, and 0.251x for 32-bit halves.

A final complete-binary screen compiled 120 GCC/link combinations and found 26 hot-loop streams. LTO, whole-program, semantic-interposition, and section-GC options left the hot loop byte-identical. An apparent IRA 1.0217x win vanished across four controlled offsets (0.985–0.995x, all intervals crossing one), and PGO/scheduling also lost. GCC 13.3's `-mtune=alderlake` remains only a real-255H A/B candidate: an AMD codegen run gave 1.0148x, but neither that host nor the available approximate port model represents the judge's heterogeneous cores.

5.4 Exact GCC 13.3 and fourth-wave search

The official `gcc:13.3.0` image was pinned by digest and used to compile the unmodified final source. The saved reproduction records the compiler, Binutils and source hashes, direct correctness results, and the exact timed-loop audit:

Build	`.text`	Timed loop	Audit/result
Default `-O3`	5,972 B	31 B / 6 insn	one helper call
BMI2 inline 700/2000	7,246 B	1,216 B / 322 insn	identical binaries
Alder tune	7,246 B	1,216 B / 322 insn	distinct schedule
Alder tune + IRA	7,199 B	1,210 B / 322 insn	third schedule

All inline loops have zero core-external calls, stack operations, and hot memory operands. All builds pass the supplied vectors and 100,000 direct random-state, random-constant one/twenty-round tests. GCC 13.3 emits the same complete binary for `-mtune=alderlake`, `raptorlake`, and `meteorlake`; it does not provide an `arrowlake` tune name.

A 34-configuration scheduler/allocator screen yielded 32 valid builds but only eight loop streams. LLVM-MCA 16's approximate hybrid model predicts 125.06 cycles for generic, 123.62 for Alder, and 121.06 for Alder+IRA. On the available AMD host, a stricter 5,000,000-call, six-warm-up, 40-sample comparison gave Alder-to-Alder+IRA paired 1.011x (CI 1.001–1.015x) on CPU 0 and 1.008x (0.998–1.025x) on CPU 4. The wrong microarchitecture and inconsistent intervals prohibit promotion, but the third stream is a high-priority 255H candidate.

The constant-placement axis also closed without a winner. Moving XOR after BSWAP or before rotate is exact for arbitrary constants but retained all 320 core operations. Historical schema-4, 24-sample campaigns on CPU 0/4 measured post-BSWAP at 0.959x/0.963x and pre-rotate at 0.965x/0.960x, with all four intervals below one. Forcing constants to memory created 160 hot memory operands and enlarged the loop to 2,015 bytes; two 21-sample campaigns measured 0.979x (CI 0.964–0.992x) and 0.988x (0.969–1.002x). Literal specialization violates the random-constant contract and still does not remove an operation. Deterministic checks show that the original, byte-swapped, inverse-rotated and top-bit-folded masks fit neither sign-extended `imm32` nor the only fully absorbable XOR masks $\{0, 2^{63}\}$.

5.5 Fifth-wave source and backend schedules

The four assignments in a two-round block are independent. A digest-pinned GCC 13.3 screen therefore compiled all 24 source orders under generic, Alder, and Alder+IRA profiles. Order $2, 1, 0, 3$ reduced LLVM-MCA's approximate generic/Alder result from 125.06/123.62 to 121.06 cycles; Alder+IRA remained 121.06. The generic/Alder loop shrank from 1,216 to 1,211 bytes while retaining 322 instructions and all 320 core operations. Its three profiles have no call, push/pop, spill, or hot memory operand and pass 100,000 random-state and random-constant tests at one and twenty rounds.

A separate 106-flag scheduler/layout screen produced nine loop streams; all 106 builds plus three source-order cross-products passed exact assembly audits. The nine shortlisted variants also passed the supplied vectors and the direct 100,000-case gate. With the approximate Alder model, `-fselective-scheduling2` and `-fno-schedule-insns2` gave 120.06 and 120.07 cycles versus 121.06 for Alder+IRA. Disabling the critical path heuristic gave 120.14; 64-byte alignment and LTO changed placement but not the 121.06-cycle loop stream. Combining source order with the first two flags did not stack the static gain. AMD diagnostics disagreed across CPUs and are not a proxy for Lion Cove or Skymont. These builds were added only to the 255H screen/confirm manifest; the incumbent remains unchanged.

5.6 Sixth-wave algebraic and lane-wise SIMD search

Smaller scalar pair blocks were checked again before changing the algorithm. Their 1/2/5/10-block loops were 143/278/646/1,211 bytes. The compact loop's initial 21-sample result was 1.035x (CI 1.007–1.043), but a six-warm-up, 6,000,000-call, 32-sample confirmation gave only 1.0004x (0.9855–1.0176). This failed reproduction prevents promotion.

We also searched for a shorter scalar identity. Model one stage as BSWAP64(ROL64(x, r) `xor` k) + a. Z3 found all 64 length-three rotate/XOR/byte-swap/add templates UNSAT for every

distinct stage. Deleting any one operation from each existing eight-operation pair and resynthesizing constants gave 32 further UNSAT results. An exhaustive linear search enumerated 4,223 length-at-most-three syntaxes, 632 distinct bit permutations, without matching a pair. These are local lower bounds for this grammar, not a global lower bound over arbitrary machine programs.

The new algorithm instead places the four independent two-round chains in four YMM lanes. A 12-variant register-residency screen found an identity-helper form that removes the last two constant loads. Exact GCC 13.3 emits 122 instructions and 579 bytes with no hot memory operand: exactly 20 each of `VPSLLVQ`, `VPSRLVQ`, `VPOR`, `VPXOR`, `VPSHUFB`, and `VPADDQ`, plus `SUB/JNE`. The cleanup itself tied the previous 124-instruction vector stream at 1.0013x (0.9983–1.0039).

AMD affinity	scalar ns	AVX2 ns	paired	95% CI
CPU 1	46.927	36.690	**1.275x**	1.260–1.292
CPU 2	34.354	36.569	**0.940x**	0.923–0.950
CPU 3	45.938	36.724	**1.248x**	1.222–1.269

Each row uses six discarded warm-ups, 32 balanced samples, 3,000,000 calls per sample, and the direct 100,000-case random-state/random-constant gate. CPU 2 reverses the CPU 1/3 ordering, demonstrating that even this VM cannot replace a 255H measurement. SSE2 was rejected at 781 instructions, 4,064 bytes, and about 0.40x. The official Core Ultra 7 255H product specification lists AVX2 as its vector extension, so AVX-512 candidates are outside the supported target contract and were not pursued. Finally, 47 extra exact-GCC13 and 53 Clang 21 target/scheduler combinations found no better static stream. Clang's spill-free 1,208-byte Arrow Lake stream scored 133.79 on the approximate Alder model, and LLVM-MCA 16 has no Lion Cove or Skymont model. The scalar build therefore remains the incumbent; the 122-instruction AVX2 build is the leading target-only candidate.

5.7 Seventh-wave split SIMD, code generation, and timing controls

The YMM implementation minimizes instruction count but retains one vector dependency chain. I therefore tested four two-XMM layouts: contiguous lanes, reversal-orbit pairs, serialized groups, and explicit lane recomputation. All passed exact GCC 13.3 auditing, supplied vectors, and 100,000 random states with random add/XOR constants. They were nevertheless clear static rejects:

Design	Instructions	Bytes	Hot memory	Alder/YMM
Contiguous XMM	242	1,349	50	2.23x
Orbit pairs	248	1,383	50	2.35x
Serialized	250	1,303	30	1.78x
Recompute lanes	288	1,509	30	1.91x

Even the best cross-architecture static ratio was 1.36x the current YMM loop. The duplicated 128-bit operations and invariant spills provide no rational basis for a noisy host campaign, so these sources remain reproducible negative experiments and are not in the 255H manifest.

A second screen compiled 113 exact GCC 13.3 and Clang 21 target, scheduler, register-allocation, alignment, and source-expression builds. One hundred complete timed loops passed exact audits, and all Pareto/source controls passed the direct 100,000-case gate. GCC's `-fira-region=one` shortened 579 bytes to 569 without changing 122 instructions, hot memory, or either model's cycle estimate. Clang emitted 548 bytes with the same instruction/model result, but it is not the judge compiler. Complementary rotate shifts are bit-disjoint, so replacing `VPOR` by `VPXOR` is exact; both compilers verified it, but only the mnemonic mix changed. A source-only inline assembly `RORX` control also left a public call under plain `gcc -O3`, so the score inline flag remains necessary. Pinning every live YMM value shortened encodings to 563 bytes, but GCC added one

hot reload and one reverse permutation, giving 124 instructions; it passed correctness but failed the performance audit.

Finally, a timing-stability tool compared separate complete processes with two 4-KiB-aligned same-process runners in alternating AB/BA order. Each sample records wall time, thread CPU time, serialized `RDTSCP`, CPU and `TSC_AUX`, context switches, and SMT-sibling load. On CPU 2, six warm-ups and 32 samples of 3,000,000 calls gave:

Control	Scalar/AVX2 paired	95% CI
Process isolated	0.8768x	0.859–0.891
Same process, page aligned	0.9225x	0.8845–0.9290

Wall, thread-CPU, and invariant-TSC ratios agreed within 0.000003; no migration or `TSC_AUX` change occurred. CPU 1/2/3 used byte-identical scalar and AVX2 binaries and normalized loops. AVX2 medians varied only 0.78%, whereas scalar medians varied 45.89%. The reversal is therefore localized to shared host scalar throughput, not code generation or timer choice. SMT load is a plausible contributor, but unavailable APERF/MPERF, cpufreq, and performance counters prevent a causal frequency/SMT claim. This diagnostic does not predict 255H performance.

5.8 Eighth-wave register allocation and target evidence

The 563-byte rejected stream suggested that low-register encoding was useful, but that fixing every live value created too much pressure. A bounded ten-case screen therefore varied allocator freedom, one versus two scratch registers, partial pinning, and encoding-aware low/high layouts. It did not brute-force the register map. Every candidate was compiled with exact GCC 13.3 and checked against the supplied vectors plus 100,000 random states and random constants at one and twenty rounds.

The winning form pins only the changing state to `ymm0`. For each rotate it first writes the right shift to one scratch, destructively left-shifts the state, and ORs the scratch back; GCC allocates all constants and controls. The result retains 122 instructions, zero hot memory, and the same 100.03/180.03 Alder/Zen proxy cycles while reducing the loop from 579 to **569 bytes**. It is a distinct normalized stream from the equally sized `-fira-region=one` build. Eight other assembly allocations emitted 124 instructions and one or two hot loads. The retained source also passes the supplied default GCC command and its official-vector harness.

A second algebraic experiment phase-staggered the two reversal orbits. With $T_j(x) = \mathrm{BSWAP}(\mathrm{ROL}(x, r_j)\ \texttt{xor}\ k_j) + a_{3-j}$, it packs $[x_0, T_3(x_3)]$ and $[x_1, T_2(x_2)]$, shares 19 immediate-rotate stages per orbit, and applies one lane epilogue. This is exact for arbitrary constants and has no hot memory, but duplicates the stream to 257 instructions and 1,253 bytes. LLVM-MCA estimated 116.04 cycles versus 100.03 on Alder, but 143.05 versus 180.03 on its Zen 2 proxy, so the cross-architecture result required measurement.

AMD affinity	569-byte assembly speedup	Phase-stagger speedup
CPU 1	0.999x (0.998–1.002)	0.758x (0.754–0.764)
CPU 3	1.000x (0.998–1.002)	0.756x (0.751–0.764)

Each row uses six discarded warm-ups, 32 balanced samples, 3,000,000 calls per sample, direct 100,000-case validation, and exact measured-binary audits. The 569-byte source is a statistical tie on both affinities. Phase staggering has about 24% lower throughput (about 32% longer runtime for the same work), directly contradicting the favorable Zen 2 scheduling proxy; it remains a reproducible negative experiment and is not in the target manifest. The first attempt also exposed a benchmark staging bug: copying a source with a quoted relative include lost its original include context. The driver now supplies the original source directory through `-iquote` to both builds, records it in those historical schema-4 files, and both campaigns above are clean reruns.

An additional exact-GCC13 backend screen tested 63 new preferred-width, load/store splitting, VEX/vzeroupper, move/store cap, cost-model, and partial/AVX tune combinations plus two references. All 65 audits passed, but every result collapsed to the two known generic/Alder normalized streams at 122 instructions, 579 bytes, and unchanged proxy cycles. Placement was not identical: (stream hash, loop-start modulo 64) yields eight classes, at generic offsets 0/8/24/40/48 and Alder offsets 8/16/48. One representative of every class passed the 100,000-case direct gate. LLVM-MCA does not model this frontend layout effect, so the seven nonbaseline representatives remain target-only rather than being rejected by this bounded flag result.

Finally, an instruction sensitivity model pins Intel's official Skymont and Crestmont packages by archive and payload hashes. The Skymont download is named for Xeon 6 E-cores, so transferring its rows to client 255H remains conditional. Two official Intel pages also conflict: Arrow Lake PerfMon calls LP-E Crestmont, while the 255H-specific ECI page calls its two LP-E cores additional Skymont cores. Crestmont is therefore retained only as a PerfMon-mapping sensitivity case. The selected AVX2 rows give the serial YMM chain a 100-cycle latency path on both tables. The scalar 80-cycle scenario is conditional: the tables omit the exact 64-bit `RORX` and the six `LEA` rows, and no official public Lion Cove per-instruction table appeared in the Intel catalog pinned on 2026-07-23. Isolated reciprocal throughput also omits mixed-port, frontend, frequency, and whole-loop effects. The model therefore documents gaps instead of selecting a 255H winner.

5.9 Ninth-wave encoding, compact blocks, and semantic timing

The next pass first strengthened what a timing sample means. The previous runner directly verified every candidate's public one- and twenty-round functions and audited the measured loop, but neither gate proved that `main` applied the twenty-round function exactly the declared number of times. A regression source that ran only half the outer iterations passed both gates and appeared 2.253x faster. Schema 5 closes this gap with an independent repeated-call oracle. Every candidate preflight, discarded warm-up, and measured process must report the declared iteration count and the exact final 256-bit state. Each 3,000,000-call campaign therefore validates 39 processes per candidate (one preflight, six warm-ups, and 32 samples), and stores the exact case set, counts, states, and canonical output hashes in a fail-closed record. Negative/zero iterations, duplicate or missing fields, hash/state mismatches, and the half-loop source are regression tests. Schema-2–4 JSON is still historical performance and assembly evidence, but does not establish this repeated-call property.

The micro-optimization exploits commutativity in three-operand VEX encoding. Swapping the two sources of `VPOR`, `VPXOR`, and `VPADDQ` keeps values and dependencies identical while placing the low-numbered changing state in the shorter ModRM position. Each of twenty `VPOR`s saves one byte. Exact GCC 13.3 consequently emits **122 instructions, 549 bytes, and zero hot memory**. Each six-instruction transform is 27 bytes, so the current `SUB`-counter form reaches the encoding bound $20 \cdot 27 + 3 + 6 = 549$ bytes. The target-only `use_incdec` control changes the outer counter and reaches 548 bytes. A digest-pinned 34-case compiler/link screen passed the supplied vectors, 100,000 random states and random add/XOR constants at one and twenty rounds, and exact audits in every case. The builds converge to five normalized streams and nine stream/alignment classes; all retain the 100.03/180.03 Alder/Zen-2 proxy path.

An independent algorithm screen explored counted blocks between one-pair loops and complete unrolling. The retained block2 processes two round-pairs per body and repeats five times. Its static timing region is only **136 bytes and 30 instructions**; the modeled non-padding dynamic count rises from 122 to 133 (134 including one alignment NOP), hot memory remains zero, and the same proxy critical path is retained. Block1 is 75 bytes/18 static and 143 modeled non-padding dynamic instructions (144 with its NOP); block5 is 321 bytes/69 static and 131 (132 with its NOP). Block5's Alder reciprocal-throughput proxy is slightly lower, 21.8 versus block2's 22.2 cycles, but that small model gain does not justify the much larger static footprint before target evidence. MCA counts deliberately exclude alignment padding and these are code-footprint

trade-offs, not speed claims. A separate exhaustive screen tried all $24 \cdot 24 = 576$ orders for the first two scalar stage-major chains and found no strict result below the tuned 120.06-cycle proxy.

The new candidates were then compared with the old 569-byte stream on CPU 1 and CPU 3, using six discarded warm-ups and 32 balanced samples of 3,000,000 calls under schema 5.

Old569/candidate	CPU 1 paired (95% CI)	CPU 3 paired (95% CI)
549-byte commutative	1.000451 (0.999294–1.001894)	1.000466 (0.998164–1.002395)
548-byte inc/dec	0.999555 (0.997869–1.000627)	1.001880 (0.997620–1.004207)
549-byte aligned	0.998679 (0.994845–1.002219)	1.000329 (0.995564–1.003437)
Block2	0.999263 (0.997061–1.001998)	0.999278 (0.997635–1.002321)

Every interval includes one. The footprint wins are reproducible, but this AMD host detects no throughput replacement. The 549/548-byte forms and block2 therefore remain target-only candidates while the adaptive scalar source stays the submission incumbent.

5.10 Tenth-wave counted front ends, code generation, and hardened timing

The previous compact blocks came from intrinsics and left avoidable frontend instructions. An exact GCC 13.3 quotient/remainder screen generated all ten pair-block sizes with conventional `DEC/JNE` control and retained three Pareto points:

AVX2 stream	Bytes	Static inst.	Modeled dynamic	Alder RThroughput
Full 20-transform inline	549	122	122	20.3
Block2 ×5	**122**	29	133	22.2
Block3 ×3 + tail1	238	53	129	21.5
Block5 ×2	292	65	127	21.2

All counts cover the complete clock-delimited loop; the dynamic column excludes alignment padding. Every stream has zero hot memory operands and the same 100.03-cycle modeled dependency path. Block2 strictly improves the older intrinsic block2's 136 bytes/30 static instructions at the same 133 dynamic instructions. Block3 also dominates the old intrinsic block5's 321 bytes/69 static/131 dynamic shape. Blocks 6–10 leave all twenty transforms static while adding loop control, and `LOOP` either exceeds its signed 8-bit reach or expands unfavorably in the Alder model. All thirteen generated decompositions passed the supplied vectors, exact binary audits, and 100,000 arbitrary-state/arbitrary-add/XOR-constant differential cases at one and twenty rounds.

A separate 29-case register, expression, and boundary screen did not lower the 549-byte floor for the current six-instruction transform. Three high-state, low-scratch register assignments tied the baseline exactly. Letting the allocator choose a high scratch register grew the loop to 569 bytes; putting the high state in ModRM grew it to 609 bytes; and over-constraining all fixed register classes produced 563 bytes, 124 instructions, and a hot constant reload. Four exact reassociations kept 122 instructions and 549 bytes. Moving the constant XOR to the right-shift branch reduced only the Zen-2 proxy from 180.03 to 160.03 cycles; Alder Lake and Meteor Lake stayed at 100.03, so it is not an Intel-target promotion. Fourteen controlled start offsets covered the important 16/32/64-byte boundaries but emitted one normalized stream; static `llvm-mca` cannot rank linked fetch-boundary effects. Every full build again passed the official vectors, 100,000 random state/constant cases, and the exact measured-loop audit.

The scalar size screen likewise found no replacement. Forced same-register `ROL` removes the only move, keeps 322 instructions, and shrinks the loop from 1,211 to 1,052 bytes, but increases the Alder/Meteor proxy from 121.06 cycles and 402 uops to 138.13 cycles and 482 uops. Same-register `SHLD` is an exact rotate, but its serial microprobe costs 3.03 modeled cycles versus 1.03 for `RORX`; it was rejected before eighty uses were expanded. These are code-size observations, not target speedups.

The three counted candidates and the 549-byte full stream were then measured on CPU 1 and CPU 3 with six discarded warm-ups, 32 balanced samples, and 3,000,000 calls per sample under the hardened schema 5. The table reports *full549/candidate* paired medians and deterministic 5,000-resample bootstrap intervals:

Candidate	CPU 1 paired (95% CI)	CPU 3 paired (95% CI)
Block2 ×5	1.002787 (0.993620–1.008628)	0.999423 (0.996616–1.002621)
Block3 ×3 + tail1	1.005277 (1.001490–1.014413)	1.001329 (0.997071–1.004260)
Block5 ×2	0.999420 (0.990163–1.008628)	1.000233 (0.997524–1.002615)

Five of the six intervals include one. CPU 1's lone block3 signal is only 0.53%, below the 1% promotion threshold, and the run is visibly nonstationary: the full stream's first seven samples are 36.07–37.44 ns before the remaining samples jump mostly into the 55–64 ns range. CPU 3 does not reproduce the signal. It is therefore diagnostic evidence, not a win. The measured binaries also passed 100,000 arbitrary-state/constant cases and exact loop audits. A separate oracle validated every preflight, warm-up, and measured process at the declared iteration count, while a fresh 128-bit nonce, campaign id, source hashes, and measured count derived a second unpredictable count for an independent semantic challenge.

Three adversarial regressions define what the added gates cover. A source that times half the calls and recomputes the other half after `clock()` fails the 0.65–1.05 inner-time/child-CPU median coverage gate. A source that times half the calls and writes the known final state afterward fails the nonce-derived alternate-count challenge. A source that merely halves the printed average fails the exact consistency check against total elapsed time and the declared count. These checks bind the recorded samples to the tested source more tightly; they do not constitute attestation against arbitrary hostile code that recognizes both generated builds, nor do they prove a specific instruction executes once per iteration. Promotion still requires source review, the exact clock-delimited assembly contract, differential tests, and independent 255H campaigns.

Consequently, no tenth-wave result displaces the adaptive scalar submission incumbent. The smaller block2, block3+tail1, and block5 frontends remain target-only alternatives that may matter on the real Core Ultra 7 255H, but neither AMD timing nor a static scheduling proxy can select among them.

5.11 Eleventh-wave stationarity gate and bounded ISA synthesis

The phase change in the preceding CPU 1 run motivated a fixed promotion gate, not a post-hoc favorable split. Each case now needs at least 16 samples, and the count must be a multiple of four times the number of cases so four predeclared contiguous blocks each preserve the complete balanced order. Absolute block-median drift may not exceed 5%; a baseline/candidate pair's block-median ratio spread may not exceed 2%; and a pair is quarantined if one block is below 0.995 while another is above 1.005. A failed candidate quarantines only that pair unless the baseline itself fails. The autotuner independently recomputes the complete record from raw samples and rejects changed summaries or thresholds. This is a conservative promotion guard, not a proof of stationarity: it can miss short excursions within a block. Barrett et al.'s changepoint study and Kalibera and Jones's repetition/effect size methodology motivate treating phase behavior and uncertainty as first-class evidence rather than assuming warm-up created a steady state. The fixed-block gate is not a reproduction of Barrett et al.'s changepoint/PELT analysis.

Fresh CPU 1 and CPU 3 campaigns used six discarded warm-ups, 24 balanced samples, and 3,000,000 calls per sample. CPU 1's full AVX2 baseline and all four AVX2 alternatives pass the absolute and pairwise gate:

CPU 1 candidate	Full549/candidate (95% CI)	Gate
Old intrinsic block2	0.999624 (0.997787–1.002890)	eligible
Counted block2	1.000750 (0.997649–1.006677)	eligible
Block3 + tail1	1.002591 (0.998829–1.006870)	eligible
Counted block5	1.002347 (0.999722–1.004974)	eligible

Every interval includes one and no eligible median reaches the 1.010 promotion threshold. CPU 1's scalar absolute block medians drift by 9.06% and its paired effect by 7.13%, so that comparison is diagnostic-only even though its aggregate ratio is large. On CPU 3 the full baseline drifts by 8.28%; every case exceeds the 5% absolute limit and several AVX2 pairs reverse by more than ±0.5%. The entire CPU 3 campaign is therefore diagnostic-only. These outcomes are stored with the raw samples in the two `stationarity_gate_timing` JSON files.

The algorithmic pass next asked whether BSWAP64($\mathrm{ROL}_r(x)$) could use only three AVX2 instructions for any contest rotation. Exhaustive enumeration plus structural exclusion established UNSAT for a bounded grammar of qword logical shifts, arbitrary fixed word-local byte selections, and XOR/OR (including carry-free ADD on disjoint fields), across all three-instruction single-input DAG shapes. More precisely, the parallel-two-unary-plus-combine shape was exhaustively enumerated, while bijectivity and projection arguments structurally exclude the other shapes. This enumeration-plus-structural exclusion is only a lower bound for that grammar, not for every x86 opcode or machine program. Three exact split-shuffle controls passed all vectors, 100,000 arbitrary-state/constant cases, and binary audits, but require **142 instructions and 669–689 bytes**, versus the incumbent's 122 instructions and 549 bytes. Moving shuffles across XOR is possible after transforming the constant; moving them across the intervening modular ADD is not, because cross-byte carries provide explicit counterexamples. Thus the two round-boundary shuffles cannot be cancelled by that reassociation.

LLVM 19 does not close the hardware gap. For every tested stream, `alderlake`, `arrowlake`, `arrowlake-s`, `lunarlake`, `meteorlake`, and `sierraforest` reported identical cycles, uops, and reciprocal throughput. The release's official `X86.td` maps the exposed `arrowlake` processor to `AlderlakePModel`; the different labels are therefore not independent 255H microarchitecture evidence. As the `llvm-mca` guide also warns, these are static scheduling-model estimates rather than measurements.

The only remaining promotion path is the guarded `probe-screen-confirm-decide` workflow. It pins a CPUID probe to every allowed logical CPU on an actual Core Ultra 7 255H, combines core type with Linux topology signals, and leaves ambiguous LP-E classification provisional. The screen must be followed by two independent confirmation sessions before `decide` can promote. A replacement requires two sessions, two physical representatives of each requested core type, matching compiler/source/manifest hashes, direct correctness, and exact measured-binary SHA-256/assembly gates; reused artifact paths or hashes cannot count as another session. Each run binds a fresh 128-bit id into its index and benchmark JSON; canonical evidence and paired-sample hashes also reject whitespace-renamed copies. A canonical fingerprint additionally pins the autotuner and benchmark drivers, loop audit, independent oracle/verifier, official archive, Python, `objdump`, and `size` executables. A changed screen protocol or mismatched confirmation protocols force a fresh measurement, and a stale topology probe cannot start a new screen. Correctness metadata must explicitly record random states and random ADD/XOR constants at both one and twenty rounds; the runner parses the verifier's exact count, seed, round, constant, and PASS output. The reference translation unit uses fixed neutral flags, while candidate flags apply only to its object and link. Source bytes are snapshotted once and both original and rewritten-performance hashes are retained. Nested topology records are validated through each cache-list element and malformed input fails closed without a traceback. Schema 5 additionally requires the repeated-call oracle, the exact case set, and all timed-process final-state records described above. An expanded balanced 15-case integration screen passed all 15 direct correctness checks and all 15 exact-binary audits; its 1,000-call timings are tool-regression evidence only. The former 19-case manifest,

including partial-unroll controls and lane-wise AVX2, passed all 19 checks and audits. Adding the distinct `-fira-region=one` and single-scratch 569-byte streams produced a 21-case smoke that passed 21/21 direct checks and audits. Adding the seven nonbaseline stream/alignment representatives produced a 28-case manifest; a fresh balanced smoke passed 28/28 direct checks and 28/28 exact-binary audits, with source-local `-iquote` context recorded for every case. Its 1,000-call provisional-host timing remains integration evidence only. The 549-byte commutative source replaces the old assembly entry; its 548-byte counter control and the old intrinsic block2 formed the 30-case schema-5 manifest. The three counted block2/block3/block5 frontends extend it to 33 target cases. All three additions passed their dedicated random-state/random-constant and exact-audit screens. Two deliberately undersized AMD/GCC12 confirmation sessions kept the incumbent while naming every missing target/compiler/sample/warm-up/iteration/random-case gate. Every P-core campaign needs paired median ≥ 1.010 and an adjusted lower bound > 1.005 without E/LP-E regression. Multiple qualifying candidates deliberately keep the incumbent pending a direct head-to-head.

6 Correctness

The following all pass:

- all 1,000 supplied one-round input/output pairs;
- the supplied 20-round pair;
- each candidate's public functions compared with the generic reference after one and twenty rounds for 100,000 deterministic random states and random add/XOR constants;
- separate complete binaries for the preserved previous and final `contest.c` files; and
- the schema-5 repeated-call and nonce-derived alternate-count final states for every measured candidate process.

```
one_round_vectors=1000
twenty_round_vector=PASS
random_differential_cases=100000
selftest=PASS

one-round testvector verification: OK (1000 pairs checked)
20-round testvector verification: OK
```

7 Score-facing repeated benchmark

An earlier microbenchmark called a `NOINLINE` 20-round function on every outer iteration, whereas GCC completely inlined the pre-optimization submission into the supplied timing loop. This changes a measurement by several percent. Also, the ratio of two separately sorted medians differed by about four percent from the median of paired sample ratios in an observed run.

The benchmark compiles complete contest files separately and can assign flags per case, including two builds of the same source. It changes only the number of calls in the supplied timing loop, discards whole-process warm-ups (six in the confirmation campaigns), pins one logical CPU, and uses cyclic/reversed case order. It retains all raw total `clock()`, printed-average, child-CPU, and external wall samples, median, MAD, p05/p95, a deterministic bootstrap median interval, paired speedup, and outlier flags in JSON. The reported per-call value is recomputed from total elapsed time, and eligible score campaigns require plausible inner-time/child-CPU coverage. Every process also reruns the official vector checks. Before timing, schema 5 compiles each `contest.c` with

only `main` renamed, links it to an independent verifier translation unit, and directly calls that candidate. This keeps the oracle self-test distinct from candidate differential verification. For every case given an `-audit-mode`, it then audits the exact performance executable against that timed-loop contract before any warm-up or measured run. The recommended commands specify a mode for every case. The same JSON records the canonical measurement-protocol fingerprint so a later decision cannot silently mix runner, verifier, archive, Python, or binary-tool revisions. A candidate requiring verifier-only flag overrides is quarantined rather than promoted. The independent repeated-call oracle then fixes the expected final state for the requested iteration count; every preflight, warm-up, and measured process must report that state exactly. A nonce-derived alternate-count build must independently reach its oracle state before timing begins.

A 10,000,000-call, 15-sample campaign produced:

Contest-shaped implementation	Median	MAD	p05–p95
Previous scalar	36.799 ns	0.924 ns	35.766–45.773 ns
Two-round unroll + BMI2	34.669 ns	2.232 ns	32.384–46.657 ns

This historical pre-optimization/final-source comparison had median paired speedup 1.068x, MAD 0.045x, and a bootstrap 95% interval of 1.008x–1.105x. A separate, noisier 20,000,000-call campaign gave 1.019x and an interval of 0.991x–1.082x. Reporting both avoids selecting only the favorable shared-VM run.

The decisive experiment instead compiled the final source both with default flags and with `-mbmi2 -finline-limit=2000`. Two independent 3,000,000-call, 21-sample campaigns produced:

Affinity	Default	Inline	Paired	Bootstrap 95% CI
CPU 0	44.467 ns	39.859 ns	**1.116x**	1.110–1.143x
CPU 4	45.924 ns	40.801 ns	**1.118x**	1.095–1.140x

The paired MAD values were 0.0281x and 0.0322x. Exact GCC 13.3 inspection has already established inline-limit 700/2000 equality and call removal. The final Intel Core Ultra 7 255H experiment therefore compares the incumbent with Alder and Alder+IRA schedules, source order $2, 1, 0, 3$, the two shortlisted backend scheduler streams, lane-wise AVX2, the old 569-byte stream, and seven nonbaseline stream/alignment representatives, plus the 549/548-byte encoding and block2 candidates in independent sessions pinned to recorded physical representatives of each core type; native tune remains diagnostic only.

8 Reproduction

From the repository root:

```
python3 solutions/benchmark_02_permutation.py \
  --case default=submissions/02/contest.c \
  --case inline=submissions/02/contest.c --baseline default \
  --case-cflag inline=-mbmi2 \
  --case-cflag inline=-finline-limit=2000 \
  --audit-mode default=default-call-allowed \
  --audit-mode inline=full-inline-320 \
  --cpu auto --iterations 3000000 \
  --warmups 3 --samples 21 --random-cases 100000 \
  --json /tmp/challenge02-inline.json

python3 solutions/02_optimization/audit_inline_02.py \
  --json /tmp/challenge02-assembly.json
python3 solutions/02_optimization/analyze_transform_lower_bound_02.py
```

```
python3 solutions/02_optimization/analyze_constant_placement_02.py \
  --emit-dir /tmp/ch2-constants/candidates \
  --json /tmp/ch2-constants/analysis.json
python3 solutions/02_optimization/reproduce_gcc133_02.py \
  --json /tmp/challenge02-gcc133.json
python3 solutions/02_optimization/screen_gcc133_schedules_02.py \
  --json /tmp/challenge02-gcc133-schedule-screen.json
python3 solutions/02_optimization/screen_gcc133_source_orders_02.py \
  --json /tmp/challenge02-gcc133-source-orders.json
python3 solutions/02_optimization/screen_gcc133_layout_02.py \
  --json /tmp/challenge02-gcc133-layout.json
python3 solutions/02_optimization/analyze_two_round_superopt_02.py \
  --json /tmp/challenge02-superopt.json
python3 solutions/02_optimization/screen_simd_02.py \
  --json /tmp/challenge02-simd.json
python3 solutions/02_optimization/screen_255h_toolchains_02.py \
  --json /tmp/challenge02-toolchains.json
python3 solutions/02_optimization/screen_split_simd_02.py \
  --output /tmp/challenge02-split-simd.json
python3 solutions/02_optimization/screen_avx2_codegen_02.py \
  --output /tmp/challenge02-avx2-codegen.json
python3 solutions/02_optimization/screen_avx2_inline_asm_alloc_02.py \
  --output /tmp/challenge02-avx2-inline-asm.json
python3 solutions/02_optimization/screen_phase_staggered_02.py \
  --output /tmp/challenge02-phase-staggered.json
python3 solutions/02_optimization/screen_avx2_commutative_layout_02.py \
  --check
python3 solutions/02_optimization/screen_avx2_pair_blocks_02.py \
  --check --jobs 4
python3 solutions/02_optimization/\
screen_tenth_avx2_counted_frontends_02.py --check
python3 solutions/02_optimization/\
screen_tenth_codegen_02.py --check
python3 solutions/02_optimization/\
screen_tenth_codegen_scalar_rotate_02.py --check
python3 solutions/02_optimization/\
screen_eleventh_isa_synthesis_02.py --check
python3 solutions/02_optimization/screen_gcc133_avx_flags_02.py \
  --json /tmp/challenge02-gcc133-avx-flags.json
python3 solutions/02_optimization/analyze_255h_instruction_model_02.py \
  --check solutions/02_optimization/instruction_model_255h_02.json
python3 solutions/benchmark_02_permutation.py \
  --case current=solutions/02_optimization/contest_simd_avx2_lanewise.c \
  --case inline_asm=solutions/02_optimization/contest_simd_avx2_inline_asm.c \
  --case phase=solutions/02_optimization/contest_simd_avx2_phase_staggered.c \
  --baseline current \
  --case-cflag current=-mavx2 \
  --case-cflag current=-DCH2_SIMD_INLINE \
  --case-cflag current=-finline-limit=2000 \
  --case-cflag inline_asm=-mavx2 \
  --case-cflag inline_asm=-DCH2_SIMD_INLINE \
  --case-cflag inline_asm=-finline-limit=2000 \
  --case-cflag phase=-mavx2 --case-cflag phase=-mbmi2 \
  --case-cflag phase=-DCH2_SIMD_INLINE \
  --case-cflag phase=-finline-limit=2000 \
  --audit-mode current=avx2-inline-lanewise \
  --audit-mode inline_asm=avx2-inline-lanewise \
  --audit-mode phase=report-only \
  --cpu 1 --iterations 3000000 --warmups 6 --samples 32 \
```

```
  --random-cases 100000 \
  --json /tmp/challenge02-eighth-cpu1.json
python3 solutions/02_optimization/benchmark_timing_stability_02.py \
  --cpu 2 --iterations 3000000 --warmups 6 --samples 32 \
  --random-cases 100000 --json /tmp/challenge02-timing-stability.json
python3 solutions/02_optimization/autotune_02_255h.py probe \
  --compiler /path/to/gcc-13.3.0 \
  --out /tmp/ch2-255h/topology.json
```

The three tenth-wave `-check` commands regenerate their static screens and compare them with the checked-in counted-frontend, code-generation, and scalar-rotate result JSON. The two hardened host campaigns are preserved separately as `solutions/02_optimization/tenth_wave_timing_02_cpu1.json` and `tenth_wave_timing_02_cpu3.json`; they retain all 32 raw sample pairs, source/protocol/binary hashes, semantic challenges, and child-CPU coverage records. Because timing and the fresh nonce are intentionally volatile, these files are measured evidence rather than byte-reproducible `-check` products. The exact six-case timing invocation is recorded in `submissions/02/README.md`. The eleventh-wave `-check` similarly verifies `eleventh_isa_synthesis_results_02.json`. Its two stationarity campaigns are preserved as `stationarity_gate_timing_02_cpu1.json` and `stationarity_gate_timing_02_cpu3.json`; the raw samples, rather than only the gate summaries, are the decision evidence.

For the longer candidate matrix:

```
python3 solutions/02_optimization/run_benchmarks.py \
  --profile native --iterations 7000000 \
  --warmup-iterations 30000000 --repeats 21 \
  --random-cases 100000
```

The 18-candidate frontend/cache sweep is reproducible with:

```
python3 solutions/02_optimization/run_deep_review_02.py \
  --iterations 2000000 --warmup-iterations 300000 \
  --samples 15 --random-cases 100000
```

The submitted source supports the supplied compatibility flags. The clean-checkout wrapper extracts the official vectors from the original archive into a temporary directory, runs the recommended score build, and removes all generated files:

```
gcc -O3 -Wall -Wextra -o /tmp/challenge02-default \
  submissions/02/contest.c
./submissions/02/run_contest.sh
```

9 References

- ISO/IEC JTC1/SC22/WG14, N1570: C11 committee draft: exact-width integers, unsigned modular arithmetic, and valid shifts.
- GCC, Byte-Swapping Builtins: the fixed reverse-map specialization.
- GCC, x86 Function Attributes: local BMI2 code generation under the supplied build command.
- GCC, GCC 13.3 Optimize Options: inline thresholds, scheduling, IRA, PGO, LTO, and loop/jump/label-alignment controls.
- de Moura and Bjørner, Z3: An Efficient SMT Solver, TACAS 2008: bit-vector equivalence and UNSAT checks for the scalar operation-deletion search.
- Solar-Lezama et al., Combinatorial Sketching for Finite Programs, ASPLOS 2006: the CEGIS-style template-and-counterexample synthesis procedure.
- LLVM, llvm-mca Machine Code Analyzer: scheduling-model throughput and resource analysis for the LLVM 16/19 proxy screens, including the model-accuracy limitation of static estimates.

- LLVM, LLVM 19.1.7 X86 processor-model definitions: the official source mapping the exposed `arrowlake` processor name to `AlderlakePModel`.
- Abel and Reineke, uops.info: Characterizing Latency, Throughput, and Port Usage of Instructions on Intel Microarchitectures, ASPLOS 2019: separating instruction count from latency, throughput, and execution-port behavior.
- Abel and Reineke, nanoBench: A Low-Overhead Tool for Running Microbenchmarks on x86 Systems: warm-up, serialization, repetition, and counter-aware low-level measurement design. Required counters were unavailable on this VM.
- Barrett, Bolz-Tereick, Killick, Mount, and Tratt, Virtual Machine Warmup Blows Hot and Cold, OOPSLA 2017: changepoint evidence that discarded warm-up alone does not guarantee a steady state. The present native-code gate uses this as methodological motivation, not as a claim that its VM/JIT model applies directly.
- Kalibera and Jones, Rigorous Benchmarking in Reasonable Time, ISMM 2013: planned repetition, uncertainty attribution, and effect-size confidence intervals.
- Intel, Intel 64 and IA-32 Optimization Reference Manual, and the Intrinsics Guide: scalar/BMI2 and AVX2 candidate analysis.
- Intel, Intel 64 and IA-32 Architectures Software Developer Manuals: VEX.vvvv, ModRM r/m, extension-bit, and instruction-length interpretation for the commutative operand-order/register-placement screen, plus the `RORX`, `ROL`, and `SHLD` semantics and encodings used by the scalar-rotate screen.
- Intel, Skymont Instruction Throughput and Latency: a Xeon 6 E-core-scoped package whose transfer to client 255H remains conditional.
- Intel, Crestmont and Redwood Cove Instruction Throughput and Latency: conditional LP-E rows if the PerfMon mapping describes the 255H execution core.
- Intel, Arrow Lake Performance Monitoring Events: Lion Cove P, Skymont E, and Crestmont LP-E mapping, one side of the official-source conflict.
- Intel, Heterogeneous Computing on Intel Core Ultra 7 255H: calls the two LP-E cores additional Skymont cores, conflicting with the PerfMon Crestmont label.
- GCC, GCC 13.3 x86 Options: AVX2 target and tuning controls, Alder Lake tuning availability, and why AMD `-march=native` data cannot stand in for the Intel target.
- Intel, Core Ultra 7 255H specifications: 6 P, 8 E, and 2 LP-E cores and AVX2 as the listed vector extension.
- Intel, Game Dev Guide for 12th Gen Intel Core Processor: hybrid CPUID flag, per-logical-CPU leaf 0x1a core type, and affinity-pinned probing.
- Linux kernel, CPU topology documentation and the CPU sysfs ABI: package/core/thread sibling and capacity cross-checks.
- Mytkowicz et al., Producing Wrong Data Without Doing Anything Obviously Wrong!: code-layout and execution-order benchmark bias.
- Claude Carlet, Boolean Functions for Cryptography and Coding Theory: ANF degree and finite-difference background for the table-separability checks.
- Google, Benchmark User Guide: warm-up, repetitions, interleaving, statistics, and machine-readable output.
- NIST, Measures of Scale: MAD as a robust dispersion report for noisy samples.